Canvas Knowledge Graph & Study Assistant

PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 · **Status** Shipped (v0.2), actively developed · **Last updated** 2026-08-03
Author Mihir Argulkar · **Repo** github.com/mihirargulkar/canvas-obsidian · **License** MIT

1. Summary

Students pay for Canvas indirectly and for an LLM directly, and the two never meet. Course material sits in Canvas as slide decks, notebooks and announcements that no assistant can read, so students paste screenshots into a chat window and get answers written in a textbook's voice rather than their professor's.

This tool syncs a student's Canvas courses into a local Obsidian vault. Slide decks are transcribed to markdown by a vision model, concepts are extracted and cross linked across lectures, and the whole thing is exposed over MCP so an existing Claude or Gemini subscription can answer questions with citations back to the specific lecture and section.

Everything stays on the student's machine as plain markdown. No hosting, no second subscription, no vendor holding the notes.

Current state: v0.2 is running daily against two live courses. 2,077 lines of Python, 45 tests, seven MCP tools, and a working end to end pipeline. Section 5 states exactly which quality claims are measured and which are not.

2. Problem

Course material is unreadable to the tools students already use. A lecture is a PPTX. A homework prompt is a PDF linked from an assignment description. A policy change is an announcement. None of it is text an LLM can be pointed at, and none of it is in one place.

Generic answers are the wrong answers. A student asking about gradient descent gets the internet's explanation. The exam tests the professor's notation, the professor's emphasis, and the example worked in week four.

The alternatives each give something up:

OPTION	WHAT IT DOES	WHAT IT COSTS
Canvas MCP servers	Broad Canvas API coverage from an LLM	No content understanding; a PPTX stays a PPTX
Hosted study assistants	RAG over uploaded files	Second subscription; course material on someone else's servers; notes you do not keep
Manual copy and paste	Works	Does not scale past one lecture; nothing accumulates

The gap: nothing turns a term of course material into a durable artifact the student owns, queryable by the assistant they already pay for.

3. Target user

Primary. An undergraduate or graduate student carrying four to six concurrent courses, who already has a Claude subscription or uses the free Gemini CLI, and who will tolerate a one time terminal setup in exchange for not paying again.

Their week: four lectures land as slide decks, two homework prompts drop, one announcement moves an exam. By midterms they cannot remember which lecture covered which topic, and searching Canvas returns filenames, not answers.

Secondary (deferred). Students who will not open a terminal. See section 12.

Explicitly not a user: institutions. Instructure's terms do not permit sharing an access token with a third party. That is fine for a student running this on their own machine and it rules out hosting it for a cohort. This is a product constraint, not an implementation detail, and it shapes every decision below.

4. Goals and non-goals

Goals

G1. Make course material answerable. Any lecture, notebook, homework prompt, announcement or syllabus should be retrievable in natural language, with a citation precise enough to go verify.

G2. Grounded, not plausible. Answers come from the student's material or the tool says it does not know. A confident wrong answer about an exam date is worse than no answer.

G3. Zero marginal cost. Run on a subscription the student already has. Transcription uses Gemini's free tier. No API bill, no hosting bill.

G4. The student owns the output. Plain markdown in a folder, openable in Obsidian, useful even if this tool disappears.

G5. Stay current without being thought about. Canvas moves all term. A daily background sync should pick up new material and say what changed.

Non-goals

NON-GOAL	WHY
Custom web UI	Obsidian already renders wikilinks, backlinks and a graph view. Building that was Phase 5 of the original plan and was cut.
Multi-user or hosted SaaS	Blocked by Instructure's terms. See section 3.
Model fine tuning	Retrieval over 1,385 chunks does not need it. Evaluated and rejected.
Doing the homework	The tool explains material. What a student does with that is between them and their integrity policy.
Broad Canvas API coverage	Grades, quizzes, submissions. Other MCP servers do this well. Content understanding is the differentiator.
Windows and Linux as first class	Code is cross platform; only macOS is tested and only macOS has the scheduled sync installer.

5. Success metrics

Two categories, kept separate on purpose: what has been measured, and what is asserted but not yet instrumented.

Measured

METRIC	TARGET	CURRENT	METHOD
Retrieval recall@5	≥ 0.90	**1.00** (10/10)	Hand labelled gold set, `tools/eval_retrieval.py`
Retrieval MRR	≥ 0.85	**0.90** sentence-transformers / **0.85** static + BM25	Same
Generic concepts excluded from graph	6/6	**6/6**	`tools/eval_graph.py`
Meaningful concept links	6/6	**2/6**	Same. See regression note below.
Fresh install footprint	< 250 MB	**~170 MB**, 74 packages	Down from 1.3 GB / 122 packages
Test suite	Green in CI	**45 passing**	GitHub Actions on every push
Re-sync cost when nothing changed	Zero model calls	**Zero**	Content hash cache, verified by run summary

The recall and MRR numbers carry a 95% confidence interval of 72 to 100%. Ten queries is a smoke test, not an instrument: one query moving one rank shifts MRR by 0.10. No fusion weights were ever tuned against it, deliberately.

Concept link quality has regressed and is the top known gap. When the extraction prompt was tuned, the gold set scored 5/6. On the current 155 concept vault it scores 2/6: “Learning Rate” is no longer extracted as a node at all, and Gradient Descent to Derivative is unreachable. Generic exclusion still holds at 6/6, so the prompt has not become noisy, it has become less complete as the corpus grew. Owner: section 12, R1.

Asserted, not yet measured

CLAIM	WHY IT IS NOT MEASURED	PLAN
Answer quality (not just retrieval)	No judged eval completed. An LLM-as-judge run exhausted Gemini’s daily quota with 63 of 70 queries unscored.	Re-run `eval_retrieval.py --judged` on a reset quota
Retrieval on realistic queries	The 70 query synthetic set exists (mean vocabulary overlap 0.15, so the queries do not parrot the source) but has never been scored end to end	Same run
Transcription fidelity	Spot checked by hand across three decks, never scored	Sample 20 slides, score against source
Time saved per student	One user, no baseline	Deferred until there are users to ask

A note on measurement discipline. Three separate bugs in this project shared one shape: *a failure was indistinguishable from a negative result*. The sync reported “no changes” when it had not looked at files. The judge counted quota errors as retrieval misses, twice, once reporting 37% recall that was really an API outage. Two published numbers (14% recall from exact source matching, 37% from the first judged run) were later found invalid and retracted. Any new metric added to this table must distinguish “measured zero” from “did not run”.

6. Requirements

Priority: **P0** ships or the product does not work; **P1** ships in v1.0; **P2** is post v1.0.

6.1 Ingestion

ID	REQUIREMENT	PRI	ACCEPTANCE
I1	Transcribe PDF, PPTX and DOCX lecture material to markdown	P0	A slide deck produces readable markdown with headings and equations preserved
I2	Read notebooks, text and markdown without a model call	P0	`.ipynb` is indexed with zero Gemini calls
I3	Ingest homework: assignment descriptions plus the prompt PDFs linked inside them	P0	“What is homework 3 asking for” answers from the prompt, not the title
I4	Ingest announcements and syllabus	P0	“Did I miss any announcements” returns posted announcements with dates
I5	Never re-transcribe unchanged content	P0	Second run reports N cached, 0 new
I6	Survive a rate limit and resume	P0	A run that hits 429 leaves prior work cached; the next run continues
I7	A cache key change must not strand existing work	P0	Adding prompt and model to the key migrates old entries rather than re-billing the corpus
I8	Images (PNG, JPG, WEBP) go straight to the vision model	P1	A scanned handout is transcribed

6.2 Knowledge graph

ID	REQUIREMENT	PRI	ACCEPTANCE
K1	Extract concepts per lecture with definitions and relationships	P0	One lecture yields concept notes with YAML frontmatter
K2	Merge concepts across lectures by canonical name	P0	A concept taught in weeks 2 and 7 is one node linked to both
K3	Exclude administrative material from the graph	P0	“Today’s Agenda” and “Course Overview” are not nodes (6/6)
K4	Keep homework and notebooks searchable but out of the graph	P0	`hw-` and `code-` prefixed notes are indexed, never extracted
K5	Output opens in Obsidian with a meaningful local graph	P0	Backlinks and graph view work with no plugins
K6	Never delete existing notes before replacements are written	P0	A mid write disk error leaves the previous vault intact

6.3 Retrieval and answering

ID	REQUIREMENT	PRI	ACCEPTANCE
R1	Deadline questions answered deterministically from the live API, never from the index	P0	“What’s due this week” matches Canvas exactly
R2	Backward looking deadline questions	P0	“Am I overdue on anything” returns past due items, not a future window
R3	Conceptual questions answered from indexed material with lecture and section citations	P0	Answer names the lecture it came from
R4	Hybrid retrieval: dense embeddings fused with BM25	P0	Exact term queries and paraphrases both work
R5	Say so when material is absent	P0	A question about an untaught topic is declined, not invented
R6	Incremental index updates	P0	Editing one note does not rebuild the index
R7	Every chunk of every note is indexed	P0	A note whose only heading is `#` still produces chunks

R7 exists because it once did not hold. Four notes produced zero chunks and were invisible to search. Fixing it recovered 48 chunks and moved MRR from 0.90 to 0.95 on the then current gold set.

6.4 Staying current

ID	REQUIREMENT	PRI	ACCEPTANCE
C1	Report what is new since the last sync	P0	Output names new announcements, assignments and files
C2	A failed read must never be persisted as “seen”	P0	A 403 during sync does not cause the next healthy run to re-report the term
C3	A freshness check must not imply “nothing posted” when files were not examined	P0	A cheap check still lists untranscribed files by name
C4	Scheduled daily sync, quiet unless something changed	P1	launchd job writes to the log only on change
C5	Cross platform scheduling	P2	Linux systemd timer, Windows Task Scheduler

C3 is a shipped bug turned requirement. A fast refresh skipped file listing, reported “no changes”, and a client concluded a lecture that was on Canvas “had not been posted”. Silence about something never checked reads as evidence.

6.5 Multiple classes

ID	REQUIREMENT	PRI	ACCEPTANCE
M1	Detect the current term’s courses without manual configuration	P0	Correct courses found across schools that never set term end dates
M2	Per class isolation: one broken class cannot sink the others	P0	A course with a restricted tab is skipped with a warning; the rest still report
M3	Every tool scopes to one class or spans all of them	P0	Optional `course` slug on all seven MCP tools
M4	One filesystem safe slug definition, used everywhere	P0	`DS 4400` resolves to `DS4400` in both the vault path and the link
M5	Cross class dashboard	P1	One file lists deadlines across all classes by date

M4 is not cosmetic. Two definitions of “slug” disagreed, so one half of the codebase wrote to `vault/DS4400/` while the other linked `[[DS/Dashboard]]`. A course named `CS1800/1802` also produced nested directories, and a malformed name could produce `../../`.

6.6 Setup and operations

ID	REQUIREMENT	PRI	ACCEPTANCE
S1	One command setup	P0	`./setup.sh` creates the venv, installs, prompts for keys, offers a first sync
S2	Re-runnable setup	P0	Running it twice is safe
S3	Install under 250 MB	P0	~170 MB measured
S4	Works with Claude Desktop, Claude Code, Gemini CLI, Cursor, VS Code	P0	One config block documented for each
S5	Missing credentials fail with instructions, not a traceback	P0	Absent `.env` prints where to get a token
S6	Library code never calls `sys.exit`	P0	Formatting a date without credentials falls back to the OS timezone

6.7 Trust and safety

ID	REQUIREMENT	PRI	ACCEPTANCE
T1	The Canvas token lives only in a gitignored `.env`	P0	Never committed, never passed through MCP
T2	All student data is gitignored	P0	`vault/`, `notes/`, `cache/`, `index/` never enter the repo
T3	Disclose that course content is sent to Gemini during transcription	P0	Stated in the README
T4	Academic integrity is the student's call, stated plainly	P0	One sentence, no lecture

7. How it works

```
Canvas API
 |
 |  ingest:  download files, LibreOffice -> PDF, Gemini vision
 v
notes/<CLASS>/*.md            faithful markdown, content-hash cached
 |
 +--> extract        -->  vault/<CLASS>/concepts/*.md   [[wikilinks]], Obsidian
 |
 +--> chunk + embed  -->  index/vectors.db              SQLite, dense + BM25
                          |
                          v
                          MCP server (stdio)  <--  live Canvas API (deadlines)
                          |
                          v
              Claude Desktop / Claude Code / Gemini CLI / Cursor
```

Pipeline. `ingest` downloads files, converts PPTX and DOCX to PDF via LibreOffice, sends vision formats to Gemini and extracts text formats directly. `extract` runs two passes: concepts per lecture, then a merge across lectures that creates the cross lecture links. `chat` chunks notes by section, embeds them, and stores vectors in SQLite. `sync` orchestrates all of it per course and diffs against seen state.

Retrieval. Dense cosine similarity over normalised vectors, fused with BM25 by reciprocal rank fusion. Deadline questions bypass all of it and hit the live API, because a stale index answering “when is the midterm” is the one failure mode with real consequences.

Live corpus (author’s machine, 2026-08-03): 2 courses, 74 notes, 1,385 indexed chunks, 155 concept nodes, 219 edges.

MCP surface

TOOL	PURPOSE	SOURCE OF TRUTH
`list_courses`	Current classes with slugs	Live API
`upcoming_assignments`	Deadlines within N days, all classes by default	Live API
`announcements`	Recent professor updates	Live API
`syllabus`	Course policies	Live API
`search_notes`	Hybrid search over lectures, homework, notebooks	Local index
`concept`	One concept node with its links	Local vault
`refresh`	What is new since the last sync	Both

Results are capped at 2,000 characters per chunk. An 800 KB chunk once exceeded the MCP 1 MB message limit and broke the transport.

8. Decision log

D1. Obsidian instead of a custom web UI. A React graph view was designed and partly built. Obsidian already ships wikilinks, backlinks, graph view, search and mobile sync, and the output is more valuable as files the student keeps than as a page only this tool can render. *Cost:* users install Obsidian. *Status:* final, and it removed an entire frontend from the project.

D2. MCP instead of a bundled chat. Students already pay for Claude or use Gemini free. Calling an API on their behalf adds a bill and a worse model. *Cost:* setup requires editing a client config. *Status:* final.

D3. SQLite and numpy instead of ChromaDB. Brute force cosine over a few thousand normalised vectors is a matrix multiply. ChromaDB brought a large dependency tree for an index that does not need one. *Cost:* linear scan. *Ceiling:* fine to roughly 100k chunks, well past a student's term.

D4. Static embeddings plus BM25 instead of PyTorch. `sentence-transformers` pulls ~800 MB of PyTorch and scores MRR 0.90 against 0.85 for static embeddings fused with BM25. Default install is the small one; if `sentence-transformers` is present it is picked up automatically. *Cost:* 0.05 MRR by default. *Status:* final, and it is what took the install from 1.3 GB to ~170 MB.

D5. Gemini for transcription, despite the client already having a model. Transcription is an unattended batch job over a hundred files. It must run on a schedule at 07:30 with nobody watching, which an interactive MCP client cannot do. *Cost:* one free API key during setup.

D6. No fine tuning. Considered and rejected. Retrieval over a term of material is a search problem; a fine tuned model would memorise one student's corpus, need retraining weekly, and lose citations.

D7. Content hash caching everywhere. The binding constraint is Gemini's free tier, not compute. Every expensive step is keyed by a hash of its inputs including the prompt and model, so a re-run is free and a prompt change correctly invalidates.

9. Risks

RISK	IMPACT	MITIGATION	STATUS
Gemini free tier quota during a large first sync	First run does not finish	Everything cached and resumable; partial results usable	Mitigated
Instructure terms forbid token sharing	No hosted version, ever	Designed as local only from the start	Accepted, shapes the product
Vision transcription errors propagate to concepts and answers	Wrong answer with a confident citation	Citations point at the source slide so a student can check	Partially mitigated; fidelity unmeasured
Concept extraction quality degrades as the corpus grows	Graph becomes less useful over a term	`eval_graph.py` catches it; it just did, at 2/6	Detected, unfixed
Canvas API changes	Sync breaks	`canvasapi` is pinned below the next major	Mitigated
Setup friction (terminal, two keys, client config)	Users drop off before first value	`setup.sh` one shot	Partially mitigated; not observed with real users
Concurrent write between the daily job and an MCP `refresh`	Corrupt cache state	None. No lockfile.	**Open**
Non-atomic `cache/*.json` writes	Crash mid write corrupts state	Corrupt state is caught and treated as empty	Partially mitigated

10. Open questions

1. **Is the graph link regression a prompt problem or a scale problem?** 5/6 to 2/6 happened as the corpus grew from 135 to 155 concepts. Needs a bisect before the prompt is touched.
2. **What is real retrieval quality?** The 70 query synthetic set has never been scored. Expectation: below the hand set, because the synthetic queries are harder.
3. **Does anyone finish setup?** Two API keys, a terminal, a client config, and optionally LibreOffice. Unknown drop off.
4. **Is one shared vault right for a whole degree?** Currently scoped to the current term. Four years of concepts might be more valuable, or unusable.
5. **Do students want the graph, or just the answers?** The graph is the expensive half of the pipeline and the least measured.

11. Dependencies and constraints

Runtime: Python 3.10+, a Canvas access token, a free Gemini API key. LibreOffice is optional and only needed for PPTX and DOCX.

Direct dependencies (7): `canvasapi`, `python-dotenv`, `google-genai`, `numpy`, `model2vec`, `mcp[cli]`, `pytest`. Majors are capped deliberately: `mcp` renamed `FastMCP` to `MCPServer` in 2.0, and an unpinned major silently breaks a fresh install.

Optional: `sentence-transformers` for +0.05 MRR at ~800 MB.

Platform: written cross platform, tested on macOS. The scheduled sync installer is launchd only.

12. Roadmap

R1. Fix concept link recall (next). Bisect the 5/6 to 2/6 regression. Exit criterion: back to 5/6 with generic exclusion still 6/6.

R2. Get a trustworthy retrieval number. Run the judged eval over all 70 synthetic queries on a fresh quota. Report with a confidence interval. Exit criterion: a number that survives the "did it actually run" check.

R3. Close the concurrency gap. A lockfile between the scheduled job and MCP `refresh`, and atomic cache writes.

R4. Reduce setup friction. Measure drop off first. Options in order of laziness: better error messages, a `doctor` command, a packaged installer.

R5. Cross platform scheduling. systemd timer and Task Scheduler.

Deferred indefinitely: hosted version (blocked by terms), custom UI (superseded by Obsidian), fine tuning (evaluated and rejected), broad Canvas API coverage (other servers do it).

Appendix A: Data model

```
notes/<CLASS>/Lecture4.md          transcribed source, YAML frontmatter
notes/<CLASS>/hw-Homework#3.md     homework prompt, excluded from the graph
notes/<CLASS>/code-gradient.md     notebook, excluded from the graph
notes/<CLASS>/announcements.md     professor updates
vault/<CLASS>/concepts/*.md        one file per concept, [[wikilinks]]
vault/<CLASS>/Dashboard.md         per class deadlines
vault/Dashboard.md                 all classes
index/vectors.db                   SQLite: chunks + embedder id
cache/                             downloads, transcriptions, seen state
```

Everything above is gitignored. A note is a lecture unless its filename is prefixed `hw-` or `code-`, or it is a poll, solution, announcement or syllabus.

Appendix B: Glossary

Chunk. A section of a note, split on `##` headings, embedded as one unit.

Slug. Filesystem safe short course key, for example `DS4400`. Derived from Canvas `course_code`, falling back to the name. Single definition in `canvas.slug_from`.

RRF (reciprocal rank fusion). Combines the dense and BM25 rankings by summing `1/(k + rank)`. Something both retrievers like outranks something only one of them found.

Recipe. A hash of the prompt plus model, mixed into every cache key so changing either correctly invalidates prior work.

MRR. Mean reciprocal rank. `1/rank` of the first correct hit, averaged. Rank 1 scores 1.0, rank 5 scores 0.2.